

Code Explanation

File Splitter Code Explanation

The provided code is a Python module that utilizes Apache Spark to split large DataFrames into multiple files based on a specified maximum number of rows per file. It generates file names based on input parameters and logs the process.

Overview of the Code

This Python function takes a Spark DataFrame and splits it into multiple CSV files, each containing a maximum number of rows specified by the user. The files are written to a specified output directory with generated file names. The function returns a list of dictionaries containing information about the generated files.

Step 1: Ensuring Output Directory Exists

The function first checks if the specified output directory exists, and if not, creates it.

```
os.makedirs(output_dir, exist_ok=True)
```

Step 2: Adding Row Numbers to the DataFrame

The function adds a unique row ID to each row in the DataFrame using the `monotonically\_increasing\_id` function from PySpark.

```
df_with_id = df.withColumn("row_id", monotonically_increasing_id())
```

Step 3: Calculating File Numbers for Each Row

The function calculates a file number for each row based on the row ID and the maximum number of rows per file.

```
df_with_file_num = df_with_id.withColumn(  
    "file_num",  
    floor(col("row_id") / max_rows_per_file) + 1  
)
```

Step 4: Getting Distinct File Numbers

The function retrieves a list of distinct file numbers from the DataFrame.

```
file_nums = df_with_file_num.select("file_num").distinct().collect()
```

Step 5: Processing Each File Partition

The function iterates over each distinct file number, filters the corresponding rows, generates a file name, and writes the data to a CSV file.

```

for row in file_nums:
    file_num = row["file_num"]
    file_df = df_with_file_num.filter(col("file_num") == file_num)
    file_df_with_name = generate_file_name(
        file_df,
        output_dir,
        file_prefix,
        batch_id,
        file_ext,
        int(file_num)
    )
    # ... rest of the processing ...

```

Step 6: Generating File Names and Writing to CSV

The function generates a file name using the `generate\_file\_name` function and writes the filtered DataFrame to a CSV file with the generated name.

```

file_name = file_df_with_name.select("FileName").first()["FileName"]
output_df = file_df_with_name.select(*[c for c in file_df_with_name.
columns
                                     if c not in ["row_id", "file_n
um"]])
output_df.write.mode("overwrite").option("header", "false").csv(
    file_name,
    sep="|"
)

```

Step 7: Collecting File Information and Logging

The function collects information about the generated files, including file names and row counts, and logs the process.

```

row_count = output_df.count()
file_info.append({
    "file_name": file_name,
    "row_count": row_count
})
logger.info(f"Generated file {file_name} with {row_count} rows")

```

Step 8: Returning File Information

The function returns a list of dictionaries containing information about the generated files.

```

return file_info

```